41

FastAPI

Serve your model as a professional API

The backend other apps and websites can call

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A Python framework for building APIs
An API	Lets other software call your model
Known for	Fast, modern, automatic docs
vs demos	For programs, not just human clicks
Your project	The backend of your Visual Defect Detector

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	**What is an API?**	The messenger idea
2	**What is FastAPI?**	A modern API framework
3	**API vs Demo Tools**	When to use which
4	**Your First API**	A few lines to an endpoint
5	**Endpoints and Methods**	GET, POST, and routes
6	**Serving an ML Model**	A real prediction endpoint
7	**Free Automatic Docs**	The standout feature
8	**Your Defect Detector**	How your project uses it
9	**Common Mistakes**	What to avoid
★	**Cheatsheet**	Quick reference

1. What is an API?

An API is a way for two pieces of software to talk to each other. One program sends a request, the other sends back a response. Think of it like a waiter in a restaurant: you (an app) give your order (a request) to the waiter (the API), who brings it to the kitchen (the server) and returns with your food (the response).

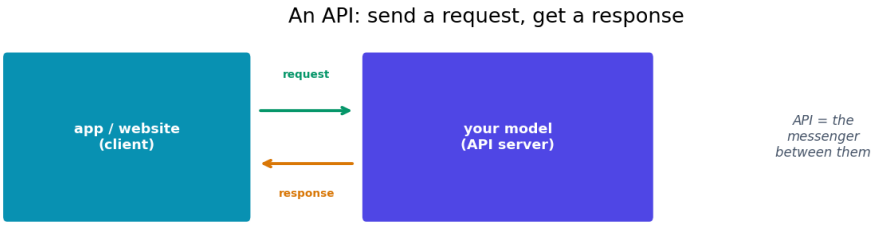

Fig 1 — An API is the messenger: a client sends a request, the server sends back a response.

For machine learning, an API lets other software use your model. A website, a mobile app, or another service sends your API some data (like an image), and your API runs the model and sends back the result (like a prediction). You have web dev experience, so you've used APIs before — now you'll build one to serve a model.

> ■ *With your MERN background, APIs are familiar territory — your React apps called backend APIs. FastAPI is how you build that backend in Python, around your ML model. Your web skills transfer directly here.*

2. What is FastAPI?

FastAPI is a modern Python framework for building APIs. As the name suggests, it's fast — both to run and to write. It's become the most popular choice for serving machine learning models because it's simple, reliable, and packed with helpful features out of the box.

You write Python functions, add a small label above each one saying what web address and method it responds to, and FastAPI turns them into a working web service. It handles all the web plumbing — receiving requests, reading data, sending responses — so you focus on your model logic.

> ■ *FastAPI is the industry standard for ML model serving. Learning it is a strong, job-relevant skill — it shows you can take a model from a notebook to a real production backend, which is exactly what ML engineering roles want.*

3. API vs Demo Tools

How does FastAPI differ from Streamlit and Gradio? Those build demos with a built-in web page for humans to click. FastAPI builds a backend service that other software calls. One is for people; the other is for programs.

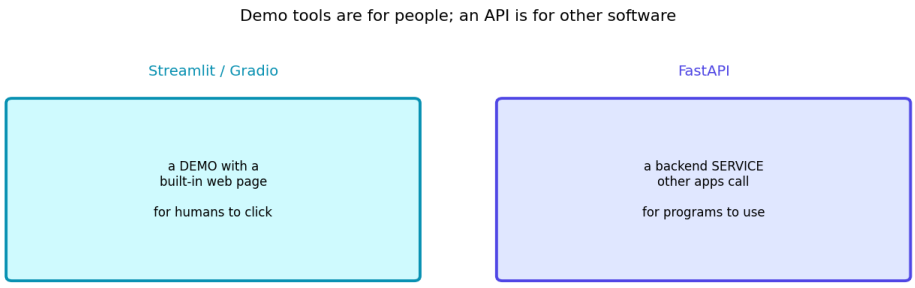

Fig 2 — Streamlit/Gradio give a human-facing demo. FastAPI gives a backend service for other software.

	Streamlit / Gradio	FastAPI
Gives you	A clickable web page	A backend service (an API)
Used by	Humans, directly	Other apps and websites
Has a UI?	Yes, built in	No — it's the engine behind a UI
Best for	Quick demos	Real apps, production, integration

They're not rivals — they're for different jobs. Use Gradio for a quick demo. Use FastAPI when you're building a real app where a separate frontend (like your React site) needs to call your model. Your Defect Detector uses exactly this split: a React frontend talking to a FastAPI backend.

4. Your First API

Here's a complete FastAPI app. Each function gets a decorator saying which web address (route) it answers.

```
# pip install fastapi uvicorn
from fastapi import FastAPI

app = FastAPI()

@app.get('/')
def home():
    return {'message': 'My ML API is running!'}

@app.get('/hello/{name}')
def hello(name: str):
    return {'greeting': f'Hello, {name}!'}
```

Run it with: **uvicorn main:app --reload**. Now visiting localhost in a browser or from another app returns the JSON responses. The @app.get('/') decorator means 'when someone requests this address, run this function'. APIs usually return JSON — clean data other programs can read easily.

■ *uvicorn is the server that runs your FastAPI app. The pattern is always: write functions with @app decorators, then run with uvicorn. Two tools, working together.*

5. Endpoints and Methods

Each web address your API responds to is called an endpoint (or route). Each endpoint uses an HTTP method that says what kind of action it is. The two you'll use most:

Method	Used for	Example
GET	Fetching data	Check if the API is alive
POST	Sending data in	Send an image to get a prediction

For an ML model, you'll mostly use POST — because the user is SENDING data (an image, some text) for the model to process. GET is for simple fetches that don't send much, like a health check. You mark each with @app.get or @app.post above the function.

```
@app.get('/health')        # GET: is the API alive?
def health():
    return {'status': 'ok'}

@app.post('/predict')      # POST: send data in to predict
def predict(data: dict):
    result = model.predict(data)
    return {'prediction': result}
```

6. Serving an ML Model

Here's the real goal: an endpoint that takes an uploaded image and returns your model's prediction. This is the heart of serving a vision model like your classifiers.

```
from fastapi import FastAPI, UploadFile
from PIL import Image
import io

app = FastAPI()
model = load_my_model()   # load ONCE at startup

@app.post('/predict')
async def predict(file: UploadFile):
    # read the uploaded image
    contents = await file.read()
    image = Image.open(io.BytesIO(contents))

    # run the model
    label, confidence = model.predict(image)

    # return JSON the frontend can use
    return {'label': label, 'confidence': float(confidence)}
```

■ *Load your model ONCE when the app starts (outside the function), not inside the endpoint. Loading it on every request would be far too slow. Same caching lesson as the demo tools — load once, reuse.*

7. Free Automatic Docs

Here's FastAPI's most loved feature. It automatically creates an interactive documentation page for your API — for free, with no extra work. Just visit /docs in your browser and you get a clickable page listing every endpoint, where you can test them live.

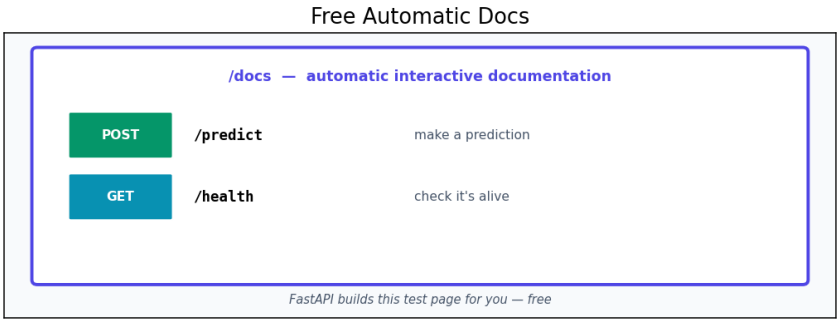

Fig 3 — FastAPI builds an interactive /docs page automatically — list every endpoint and test it live.

This is incredibly useful. You can test your prediction endpoint by uploading an image right in the browser, with no separate tool. It also serves as instant documentation for anyone using your API. This auto-docs feature is a big reason FastAPI is so popular — you get professional API documentation for nothing.

8. Your Defect Detector

Your Visual Defect Detector project already uses this exact architecture. Let's see how the pieces you've learned fit together in your real project.

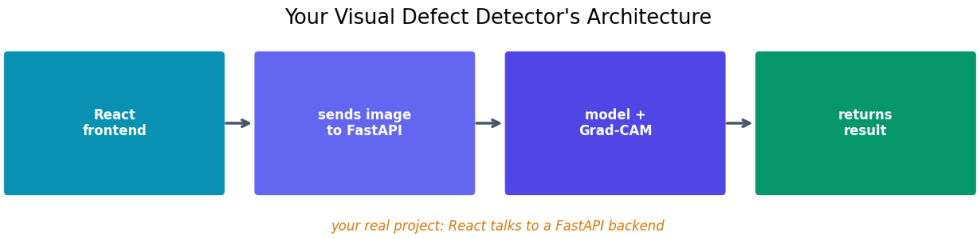

Fig 4 — Your Defect Detector: a React frontend sends an image to a FastAPI backend running your model.

The flow: your React frontend lets a user upload an image and sends it to your FastAPI backend. FastAPI receives the image, runs your EfficientNet-B0 model, generates the Grad-CAM heatmap, and returns the result as JSON. React then displays the prediction and heatmap. This clean frontend-backend split, with FastAPI serving the model, is professional, production-style design — a strong portfolio piece.

> ■ *This separation — React frontend, FastAPI model backend — is exactly how real ML products are built. Your Defect Detector already demonstrates this. It's worth highlighting in interviews: it shows end-to-end, production-minded skills.*

9. Common Mistakes

Mistake	Why it's a problem	Fix
Loading model per request	Painfully slow	Load once at startup
Wrong method (GET vs POST)	Can't send data properly	POST for sending data in
Returning non-JSON types	Response errors	Return dicts / JSON-safe values
Forgetting uvicorn	API won't run	Run with uvicorn main:app
No error handling	Crashes on bad input	Validate and handle errors
Blocking slow code	Ties up the server	Use async / background tasks

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
API	software talking to software
FastAPI	Python framework for building APIs
Endpoint	an address your API responds to
GET	fetch data (health check)
POST	send data in (predict)
Install	pip install fastapi uvicorn
Create app	app = FastAPI()
GET route	@app.get('/path')
POST route	@app.post('/predict')
File upload	async def f(file: UploadFile)
Run server	uvicorn main:app --reload
Auto docs	visit /docs in the browser
Load model	once at startup, not per request

The one idea to remember: FastAPI turns your model into an API — a backend service other software can call. You write Python functions with @app decorators, run with uvicorn, and get free interactive docs. It's the professional way to serve ML models, exactly like your Defect Detector's backend.

Next Topic: Docker — how to package your app and everything it needs into a 'container' that runs the same anywhere. One of your key skill-gap targets, and the standard for shipping real software.